

Semantic Analysis

a.y. 2025-2026

1 / 85

Semantic Analysis

- Compute additional information once the syntactic structure is known
- Information that is beyond the capabilities of context-free grammars
- Typically
 - Populate symbol tables after declarations (e.g., which is the type of declared variable?)
 - Perform type inference and type checking on expressions and statements
- Two categories of analysis:
 - Analysis required to establish correctness
 - Analysis to enhance efficiency of the translated program

2 / 85

Semantic Analysis

- Handiest method to describe semantic analysis is to identify properties (**attributes**) of grammar symbols and to write rules (**semantic rules**) to describe how the computation of those properties is related to the grammar productions
- This is what we call **attributed grammar**, or **syntax-directed definition**
- Instead of the derivation tree, a better basis for semantic computations is the **abstract syntax tree** (a compact representation of the derivation tree). However, the abstract syntax tree can itself be defined by an appropriate instance of semantic analysis

3 / 85

Semantic Analysis

WHAT NEXT

- Syntax-Directed Definitions
- Techniques for the computation of the specified attributes

4 / 85

Syntax-Directed Definitions

- Syntax-directed definitions (SDDs) are context-free grammars enriched with attributes and rules to compute them
 - **Attributes:** Associated with grammar symbols; can be numbers, types, references to the symbol table, location of a variable in memory, object code of a procedure, etc.
 - **Semantic rules:** Associated with productions; typically induce the computation of some attributes as functions of the attributes of other symbols of the production
- Used to perform semantic analysis and more

5 / 85

Attributes

Attributes of non-terminals are classified as

- **Synthesized:** attributes of the driver defined as a function of the attributes of symbols in the production
- **Inherited:** attributes of non-terminals in the body defined as a function of the attributes of symbols in the production

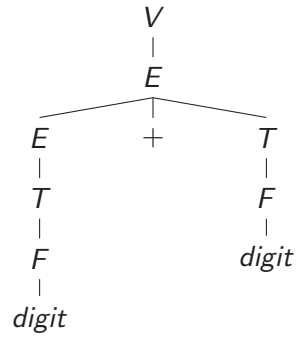
Attributes of terminals can only be synthesized: They are supplied by the lexical analyzer, there is no rule to compute them

6 / 85

SDD for arithmetic expressions

EXAMPLE

Consider the following parse tree for the LALR(1) grammar of arithmetic expressions



If one *digit* has lexical value 3 and the other *digit* has lexical value 4, then the value of the expression should be 7

7 / 85

SDD for arithmetic expressions

EXAMPLE

$V \rightarrow E$	$\{V.val = E.val\}$
$E \rightarrow E_1 + T$	$\{E.val = E_1.val + T.val\}$
$E \rightarrow T$	$\{E.val = T.val\}$
$T \rightarrow T_1 * F$	$\{T.val = T_1.val * F.val\}$
$T \rightarrow F$	$\{T.val = F.val\}$
$F \rightarrow (E)$	$\{F.val = E.val\}$
$F \rightarrow digit$	$\{F.val = digit.lexval\}$

8 / 85

Evaluation of SDDs

Define a **dependency graph** for the SDD:

- Set a node of the dependency graph for each attribute associated with each parse tree node
- For each attribute $X.x$ used to define the attribute $Y.y$, set an edge from the node for $X.x$ to the node for $Y.y$.
The edge means “ $X.x$ **is needed to define** $Y.y$ ”

The SDD can be evaluated after any **topological sort** of the dependency graph.

To get a topological sort, number the nodes $N_1, \dots, N_k$ of the dependency graph in such a way that if there is an edge from N_i to N_j then $i < j$.

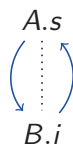
9 / 85

Evaluation of SDDs

When the SDD has both synthesized and inherited attributes there is no guarantee that a topological sort (and hence an evaluation order) exists

There might be a cycle in the dependency graph, hence no topological sort:

$$A \rightarrow B \quad \{A.s = B.i; B.i = A.s + 7\}$$



10 / 85

Evaluation of SDDs

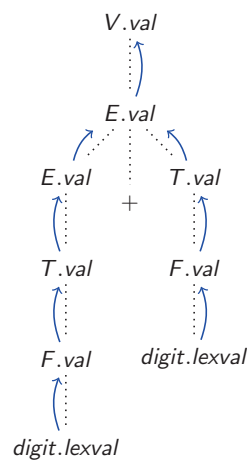
Classes of SDDs for which the existence of a topological sort of the dependency graph is guaranteed:

- **S-attributed SDDs:** Only synthesized attributes, can just use post-order to evaluate
- **L-attributed SDDs:** Either synthesized attributes, or inherited attributes such that
 - For each production $A \rightarrow X_1 \dots X_n$
 - The definition of each $X_j.i$ uses at most
 - Inherited attributes of A
 - Inherited or synthesized attributes of the left siblings $X_1, \dots, X_{j-1}$

11 / 85

Example: LALR(1) Arithmetic Expressions

DEPENDENCY GRAPH



12 / 85

Example: LALR(1) Arithmetic Expressions

EVALUATION OF SDD

- Computing a topological sort of the nodes is superfluous
- A **post-order visit** of the parse tree is enough to evaluate the SDD

This is always the case for S-attributed SDDs

```
post-order(N) {
  foreach child C of N do post-order(C);
  evaluate attributes of N;
}
```

13 / 85

Example: LALR(1) Arithmetic Expressions

EVALUATION OF SDD

```
procedure expEval(N)
  foreach child C of N do
    expEval(C) ;
  if N is a Fnode then
    if N has only one child then
      | assign to N.val the attribute digit.lexval of its child
    else
      | assign to N.val the attribute of its second child
  else if N is a Enode then
    if N has only one child then
      | assign to N.val the attribute of its child
    else
      | assign to N.val the sum of the attributes of its first
      | and third children
  ...
```

14 / 85

Example: Typed Declarations

Sometimes synthesized attributes are not appropriate

Consider the following grammar for variable declaration:

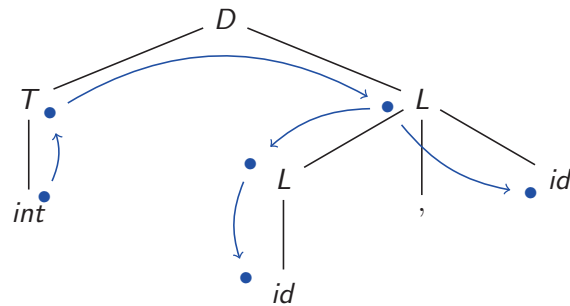
$$\begin{aligned} D &\rightarrow TL \\ T &\rightarrow int \\ T &\rightarrow float \\ L &\rightarrow L, id \\ L &\rightarrow id \end{aligned}$$

Design semantic actions to assign the declared type (either “integer” or “float”) to the attribute *id.type*

Example: Typed declarations

$$\begin{aligned} D &\rightarrow TL && \{L.type = T.type\} \\ T &\rightarrow int && \{T.type = integer\} \\ T &\rightarrow float && \{T.type = float\} \\ L &\rightarrow L_1, id && \{L_1.type = L.type; id.type = L.type\} \\ L &\rightarrow id && \{id.type = L.type\} \end{aligned}$$

Example: Typed declarations



One single inherited attribute and what about evaluation now?

Either pre-compute a topological sort, or apply an ad hoc mix of in-order and preorder

17 / 85

Example: Typed declarations

EVALUATION OF SDD

```

procedure typeEval(N)
  if N is a Dnode then
    typeEval(Tnode child of N) ;
    assign the type of the Tnode child to the Lnode child ;
    typeEval(Lnode child of N);
  else if N is a Lnode then
    assign N.type to the IDnode child ;
    if N has a Lnode child then
      assign N.type to the Lnode child ;
      typeEval (Lnode child of N) ;
  else if N is a Tnode then
    assign integer to N.type if child of N is int, assign float to
    N.type otherwise
  
```

18 / 85

Example: LL(1) arithmetic expressions

Different grammars pose different challenges in designing SDDs

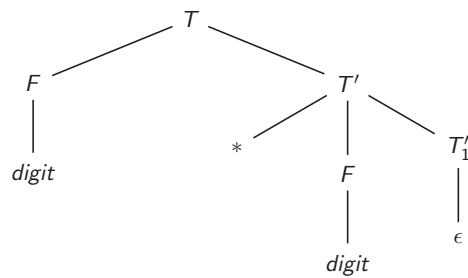
Consider the LL(1) grammar for arithmetic expressions:

$$\begin{aligned} V &\rightarrow E \\ E &\rightarrow TE' \\ E' &\rightarrow +TE' \mid \epsilon \\ T &\rightarrow FT' \\ T' &\rightarrow *FT' \mid \epsilon \\ F &\rightarrow (E) \mid \textit{digit} \end{aligned}$$

19 / 85

Example: LL(1) arithmetic expressions

The parse tree for $3 * 5$ has the subtree



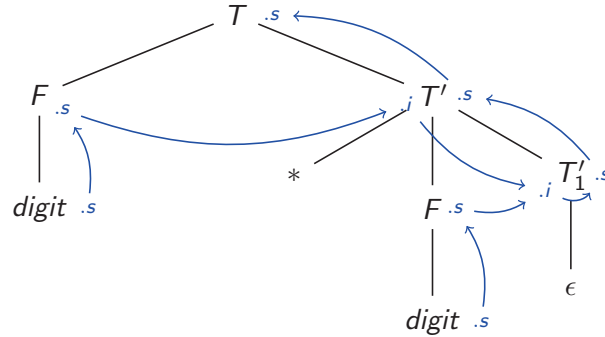
Here one *digit* is on the left subtree of the root, while the operator $*$ and the other digit are on the right subtree of the root

How can we get the expected $T.val = 15$?

20 / 85

Example: LL(1) arithmetic expressions

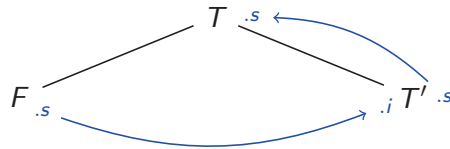
Both inherited and synthesized attributes together



Where every $.s$ stands for a synthesized attribute of the node, and every $.i$ stands for an inherited attribute of the node

21 / 85

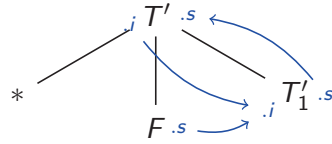
Example: LL(1) arithmetic expressions



$$T \rightarrow FT' \quad \{T.s = T'.s; T'.i = F.s\}$$

22 / 85

Example: LL(1) arithmetic expressions



$$T' \rightarrow *FT'_1 \quad \{T'.s = T'_1.s; T'_1.i = T'.i * F.s\}$$

23 / 85

Example: LL(1) arithmetic expressions

L-attributed SDD for arithmetic expressions

$V \rightarrow E$	$\{V.s = E.s\}$
$E \rightarrow TE'$	$\{E.s = E'.s; E'.i = T.s\}$
$E' \rightarrow +TE'_1$	$\{E'.s = E'_1.s; E'_1.i = E'.i + T.s\}$
$E' \rightarrow \epsilon$	$\{E'.s = E'.i\}$
$T \rightarrow FT'$	$\{T.s = T'.s; T'.i = F.s\}$
$T' \rightarrow *FT'_1$	$\{T'.s = T'_1.s; T'_1.i = T'.i * F.s\}$
$T' \rightarrow \epsilon$	$\{T'.s = T'.i\}$
$F \rightarrow (E)$	$\{F.s = E.s\}$
$F \rightarrow digit$	$\{F.s = digit.lexval\}$

24 / 85

Evaluation during bottom-up parsing

The challenge is to implement the translation during parsing (vs obtaining the parse tree, then annotating it, then evaluating the annotated parse tree)

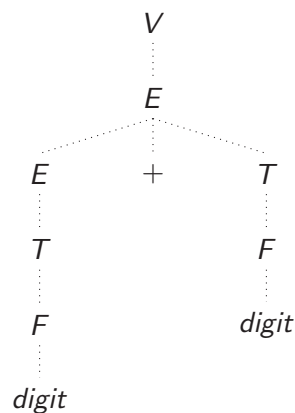
By far the simplest implementation occurs when the grammar can be parsed by the shift/reduce algorithm and the underlying SDD is S-attributed

25 / 85

Evaluation during bottom-up parsing

EXAMPLE

Take the LALR(1) grammar for arithmetic expression and consider the sequence of reductions for parsing the string $3 + 4$ (complete derivation tree in the background)

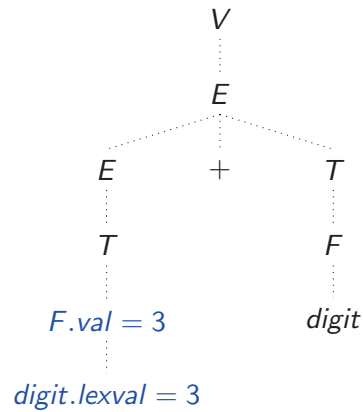


26 / 85

Evaluation during bottom-up parsing

EXAMPLE

$$F \rightarrow \text{digit} \quad \{F.val = \text{digit.lexval}\}$$

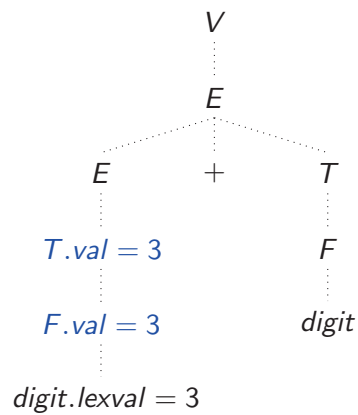


27 / 85

Evaluation during bottom-up parsing

EXAMPLE

$$T \rightarrow F \quad \{T.val = F.val\}$$

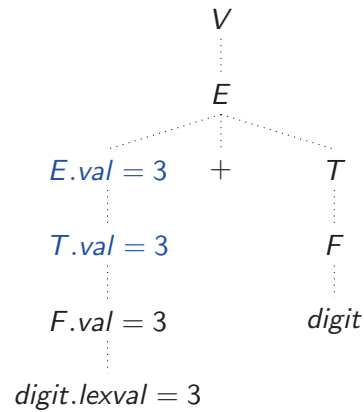


28 / 85

Evaluation during bottom-up parsing

EXAMPLE

$$E \rightarrow T \quad \{E.val = T.val\}$$

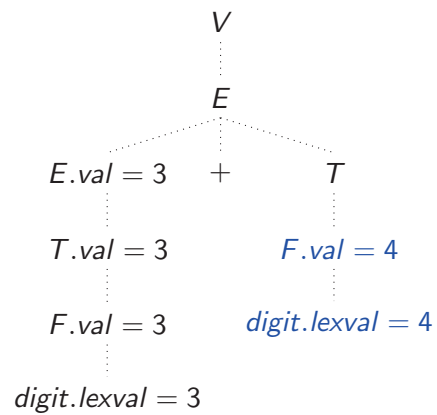


29 / 85

Evaluation during bottom-up parsing

EXAMPLE

$$F \rightarrow digit \quad \{F.val = digit.lexval\}$$

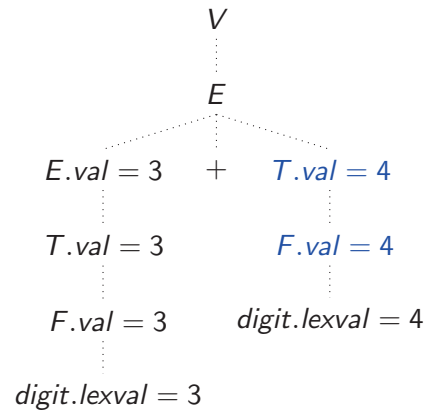


30 / 85

Evaluation during bottom-up parsing

EXAMPLE

$$T \rightarrow F \quad \{T.val = F.val\}$$

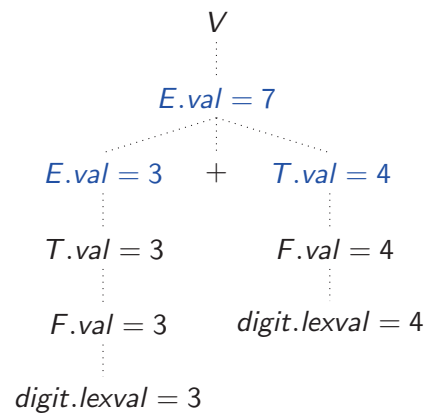


31 / 85

Evaluation during bottom-up parsing

EXAMPLE

$$E \rightarrow E_1 + T \quad \{E.val = E_1.val + T.val\}$$

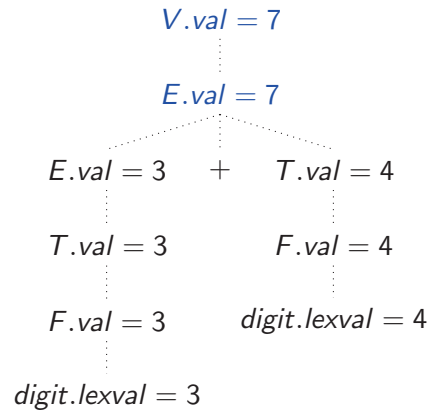


32 / 85

Evaluation during bottom-up parsing

EXAMPLE

$$V \rightarrow E \quad \{V.val = E.val\}$$



33 / 85

Evaluation during bottom-up parsing

Revisiting the shift/reduce algorithm for S-attributed SDDs:

- Together with the state stack *stSt* and the symbol stack *symSt*, use a semantic stack *semSt* to keep (records of) attributes
- Execute the code associated with $A \rightarrow \beta$ when the production is reduced
- The needed attributes of the symbols on top of *symSt* are found at the corresponding positions on top of *semSt*

34 / 85

Evaluation during bottom-up parsing

EXAMPLE

Run the shift/reduce algorithm on input "*digit + digit*" for the LALR(1) grammar of arithmetic expressions

<i>digit + digit</i> \$	<i>stSt</i>	0		
	<i>symSt</i>			
	<i>semSt</i>			
<u><i>digit</i></u> <i>+digit</i> \$	<i>stSt</i>	0	1	
	<i>symSt</i>	<i>digit</i>		
	<i>semSt</i>	3		push <i>digit.lexval</i>
<u><i>digit</i></u> <i>+digit</i> \$	<i>stSt</i>	0	6	
	<i>symSt</i>	<i>F</i>		reduce $F \rightarrow digit$
	<i>semSt</i>	3		pop <i>digit.lexval</i> ; push <i>F.val</i>

35 / 85

Evaluation during bottom-up parsing

EXAMPLE

<u>digit</u> +digit\$	stSt	0	5		
	symSt	T		reduce $T \rightarrow F$	
	semSt	3		pop $F.val$; push $T.val$	
<u>digit</u> +digit\$	stSt	0	4		
	symSt	E		reduce $E \rightarrow T$	
	semSt	3		pop $T.val$; push $E.val$	
<u>digit</u> +digit\$	stSt	0	4	9	
	symSt	E	+		
	semSt	3	0		
				push a dummy number to keep stacks aligned	
<u>digit</u> + digit\$	stSt	0	4	9	1
	symSt	E	+	digit	
	semSt	3	0	4	
					push $digit.lexval$

36 / 85

Evaluation during bottom-up parsing

EXAMPLE

<u>digit + digit</u> \$	stSt	0	4	9	6	
	symSt	E	+	F		reduce $F \rightarrow digit$
	semSt	3	0	4		pop $digit.lexval$; push $F.val$
<u>digit + digit</u> \$	stSt	0	4	9	12	
	symSt	E	+	T		reduce $T \rightarrow F$
	semSt	3	0	4		pop $F.val$; push $T.val$
<u>digit + digit</u> \$	stSt	0	4			
	symSt	E				reduce $E \rightarrow E + T$
	semSt	7				pop $T.val$; pop; pop $E_1.val$; push $E.val = E_1.val + T.val$
<u>digit + digit</u> \$	stSt	0	1			
	symSt	V				reduce $V \rightarrow E$; Accept
	semSt	7				$print(7)$

37 / 85

Case study

FROM STRINGS TO NUMBERS: G0

Goal: translate strings of digits to their decimal number values

$S \rightarrow Digits$	$\{print(Digits.v)\}$
$Digits \rightarrow Digits_1 d$	$\{Digits.v = Digits_1.v * 10 + d.lexval\}$
$Digits \rightarrow d$	$\{Digits.v = d.lexval\}$

The grammar is LALR(1), the SDD is S-attributed, attributes can be easily evaluated

38 / 85

Case study

FROM STRINGS TO NUMBERS

Goal: if the string of digits is prefixed by the terminal o then translate it to its octal number value, otherwise translate it to its decimal number value

39 / 85

Case study

FROM STRINGS TO NUMBERS: G1

First attempt: simply add one production to G0 to generate strings of digits prefixed by o

$$\begin{aligned} S &\rightarrow Num \\ Num &\rightarrow o\ Digits \mid Digits \\ Digits &\rightarrow Digits\ d \mid d \end{aligned}$$

The grammar is LALR(1)

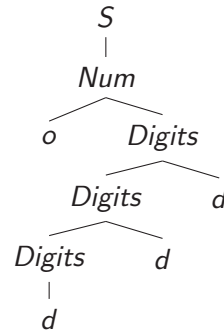
Can we compute number values using synthesized attributes?

40 / 85

Case study

FROM STRINGS TO NUMBERS: G1

$S \rightarrow Num$
 $Num \rightarrow o\ Digits \mid Digits$
 $Digits \rightarrow Digits\ d \mid d$



The information required to get the translation right is not available from below, cannot be synthesized

Bad luck, change the grammar

41 / 85

Case study

FROM STRINGS TO NUMBERS: G2

Second attempt: clone productions to separate the management of octal numbers from that of decimal numbers

$S \rightarrow Num$	$\{print(Num.v)\}$
$Num \rightarrow o\ O$	$\{Num.v = O.v\}$
$Num \rightarrow D$	$\{Num.v = D.v\}$
$O \rightarrow O_1\ d$	$\{O.v = O_1.v * 8 + d.lexval\}$
$O \rightarrow d$	$\{O.v = d.lexval\}$
$D \rightarrow D_1\ d$	$\{D.v = D_1.v * 10 + d.lexval\}$
$D \rightarrow d$	$\{D.v = d.lexval\}$

LALR(1), good for synthesising, but redundant

42 / 85

Case study

FROM STRINGS TO NUMBERS: G3

Third attempt: introduce extra unit productions to force semantic actions

$S \rightarrow Num$	$\{print(Num.v)\}$
$Num \rightarrow Octal\ Digits$	$\{Num.v = Digits.v\}$
$Num \rightarrow Decimal\ Digits$	$\{Num.v = Digits.v\}$
$Octal \rightarrow o$	$\{base = 8\}$
$Decimal \rightarrow \epsilon$	$\{base = 10\}$
$Digits \rightarrow Digits_1\ d$	$\{D.v = D_1.v * base + d.lexval\}$
$Digits \rightarrow d$	$\{Digits.v = d.lexval\}$

LALR(1), but uses the global variable *base*

Case study

FROM STRINGS TO NUMBERS: G4

Fourth attempt: let the appropriate base be captured by a synthesized attribute

$S \rightarrow D$	$\{print(D.v)\}$
$D \rightarrow D\ d$	$\{D.v = D_1.v * D_1.base + d.lexval;$ $D.base = D_1.base\}$
$D \rightarrow B\ d$	$\{D.v = d.lexval;$ $D.base = B.val\}$
$B \rightarrow o$	$\{B.val = 8\}$
$B \rightarrow \epsilon$	$\{B.val = 10\}$

Another case study

CONVERSION FROM INTEGERS TO FLOATS

Take the following grammar

$$\begin{aligned} E &\rightarrow E \text{ dop } T \mid T \\ T &\rightarrow \text{num} \mid \text{num.num} \end{aligned}$$

Goal: add semantic actions to compute the value of the expression so that:

- If the expression contains only integer numbers (terminal *num*) then *dop* is translated to the integer division operator *div*
- If the expression contains at least a real number (terminal *num.num*) then the whole expression is converted to a floating point expression, with integers promoted to floating-point numbers. For example *5 dop 2 dop 2.0* is converted to *float(5) / float(2) / 2.0*

45 / 85

Another case study

CONVERSION FROM INTEGERS TO FLOATS

We need to know whether the parse tree has any *num.num* leaf, and we can compute the value of the expression only when we know whether the operands have to be promoted to floating-point or not.

Then:

- We use a boolean attribute *float* which flows from the leaves to the root (synthesized attribute)

$$E \rightarrow E_1 \text{ dop } T \quad E.\text{float} = E_1.\text{float} \text{ OR } T.\text{float}; \dots$$

$$E \rightarrow T \quad E.\text{float} = T.\text{float}; \dots$$

$$T \rightarrow \text{num} \quad T.\text{float} = \text{false}; \dots$$

$$T \rightarrow \text{num.num} \quad T.\text{float} = \text{true}; \dots$$

46 / 85

Another case study

CONVERSION FROM INTEGERS TO FLOATS

- If the tree has any *num.num* leaf, then the *float* attribute of the root is *true*. Once we know the *float* attribute of the root, we can decide the type of the whole expression and let it flow towards the leaves through the attribute *type* (inherited attribute)
- To be sure that we decide the type of the expression just at the root of the parse tree (rather than at some *E*-node which cannot access all the leaves of the tree), we add the extra production

$$S \rightarrow E$$

to distinguish the *E*-node playing the root. The *type* attribute for *E* is set at this extra production

Another case study

CONVERSION FROM INTEGERS TO FLOATS

$S \rightarrow E$	$E.type = \text{IF } E.float \text{ THEN } float \text{ ELSE } int; \dots$
$E \rightarrow E_1 \text{ dop } T$	$E_1.type = E.type; T.type = E.type; \dots$
$E \rightarrow T$	$T.type = E.type; \dots$
$T \rightarrow num$	$\dots$
$T \rightarrow num.num$	$\dots$

Another case study

CONVERSION FROM INTEGERS TO FLOATS

- The value of the expression, which depends on its *type*, is computed in the attribute *val* (synthesized attribute)

$S \rightarrow E$	$S.val = E.val; \dots$
$E \rightarrow E_1 \text{ dop } T$	$E.val = \text{IF } E.type = int$ $\quad \text{THEN } E_1.val \text{ div } T.val$ $\quad \text{ELSE } E_1.val / T.val; \dots$
$E \rightarrow T$	$E.val = T.val; \dots$
$T \rightarrow num$	$T.val = \text{IF } T.type = int$ $\quad \text{THEN } num.lexval$ $\quad \text{ELSE } float(num.lexval); \dots$
$T \rightarrow num.num$	$T.val = num.num.lexval; \dots$

49 / 85

Another case study

CONVERSION FROM INTEGERS TO FLOATS

And now:

Collect all the semantic actions and get the SDD

Get the annotated parse tree

Evaluate

50 / 85

Abstract syntax trees

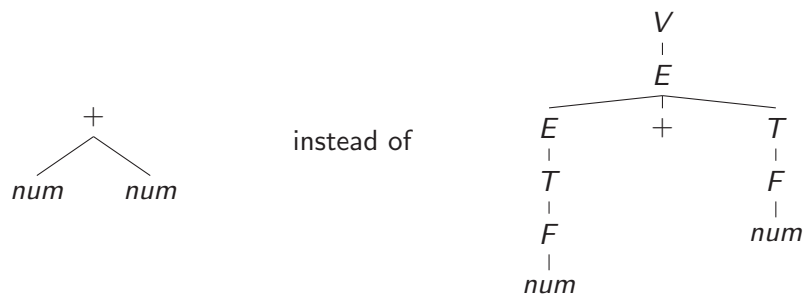
Compact representations of parse trees, often taken as intermediate representation

No general rule to define them, it depends on the grammar at hand and on the implementation choice

Surely they must contain all the information needed to carry on the analysis

51 / 85

Abstract syntax trees



52 / 85

Abstract syntax trees

Provide an S-attributed grammar so that the abstract syntax tree can be constructed while parsing

Assumptions:

- Function *newLeaf*(*label*, *val*) creates a leaf object with two fields: the label of the node and its value
- Function *newNode*(*label*, $c_1, \dots, c_k$) creates an internal node with k children: *label* is the label of the node, and $c_1, \dots, c_k$ are references to the children nodes

53 / 85

Abstract syntax trees

ARITHMETIC EXPRESSIONS: LALR GRAMMAR

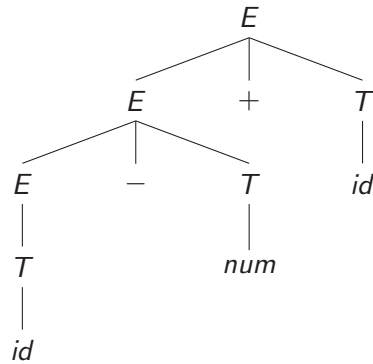
$E \rightarrow E_1 + T$	$\{E.node = newNode(+, E_1.node, T.node)\}$
$E \rightarrow E_1 - T$	$\{E.node = newNode(-, E_1.node, T.node)\}$
$E \rightarrow T$	$\{E.node = T.node\}$
$T \rightarrow (E)$	$\{T.node = E.node\}$
$T \rightarrow id$	$\{T.node = newLeaf(id, id.entry)\}$
$T \rightarrow num$	$\{T.node = newLeaf(num, num.lexval)\}$

54 / 85

Abstract syntax trees

ARITHMETIC EXPRESSIONS: LALR GRAMMAR

Parse $id - num + id$ resulting from lexing $a - 4 + c$



Reductions:

- ① $T \rightarrow id$
- ② $E \rightarrow T$
- ③ $T \rightarrow num$
- ④ $E \rightarrow E - T$
- ⑤ $T \rightarrow id$
- ⑥ $E \rightarrow E + T$

55 / 85

Abstract syntax trees

ARITHMETIC EXPRESSIONS: LALR GRAMMAR

First reduction:

$T \rightarrow id$ $\{ T.node = newLeaf(id, id.entry) \}$



56 / 85

Abstract syntax trees

ARITHMETIC EXPRESSIONS: LALR GRAMMAR

Second reduction:

$$E \rightarrow T \quad \{E.node = T.node\}$$



57 / 85

Abstract syntax trees

ARITHMETIC EXPRESSIONS: LALR GRAMMAR

Third reduction:

$$T \rightarrow num \quad \{T.node = newLeaf(num, num.lexval)\}$$



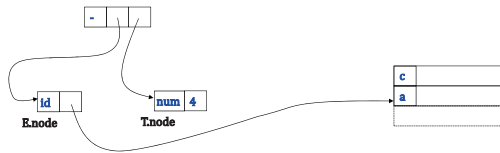
58 / 85

Abstract syntax trees

ARITHMETIC EXPRESSIONS: LALR GRAMMAR

Fourth reduction:

$$E \rightarrow E_1 - T \quad \{E.node = newNode('-', E_1.node, T.node)\}$$



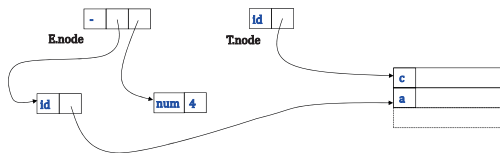
59 / 85

Abstract syntax trees

ARITHMETIC EXPRESSIONS: LALR GRAMMAR

Fifth reduction:

$$T \rightarrow id \quad \{T.node = newLeaf(id, id.entry)\}$$



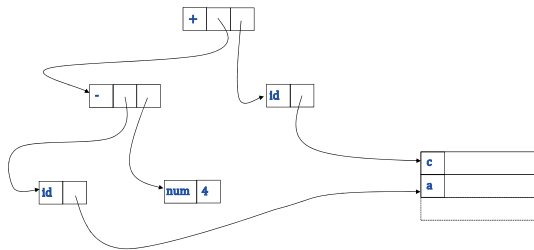
60 / 85

Abstract syntax trees

ARITHMETIC EXPRESSIONS: LALR GRAMMAR

Last reduction:

$$E \rightarrow E_1 + T \quad \{E.node = newNode('+', E_1.node, T.node)\}$$



61 / 85

Abstract syntax trees

ARITHMETIC EXPRESSIONS: LL GRAMMAR

$$\begin{aligned} E &\rightarrow TE' \\ E' &\rightarrow +TE'_1 \\ E' &\rightarrow -TE'_1 \\ E' &\rightarrow \epsilon \\ T &\rightarrow (E) \\ T &\rightarrow id \\ T &\rightarrow num \end{aligned}$$

62 / 85

Abstract syntax trees

ARITHMETIC EXPRESSIONS: LL GRAMMAR

We use synthesized and inherited attributes after a strategy similar to the case of the computation of the value of the expression

This time, rather than values, we pass around references to nodes

We get the annotated parse tree

Then we define the dependency graph and find a topological sort of its nodes

Last, evaluation constructs the abstract syntax tree

63 / 85

Abstract syntax trees

ARITHMETIC EXPRESSIONS: LL GRAMMAR

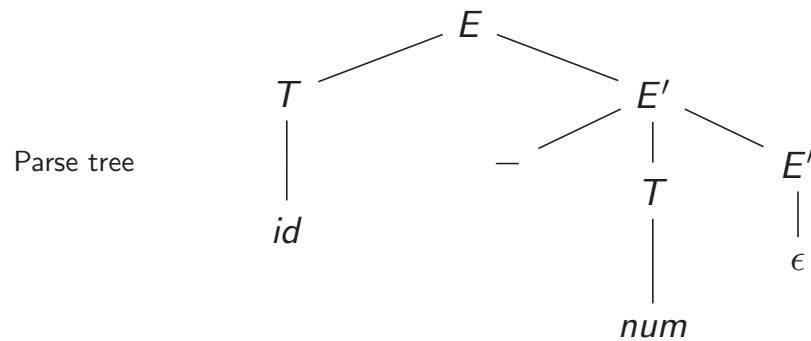
$E \rightarrow TE'$	$\{E.node = E'.node;$ $E'.i = T.node\}$
$E' \rightarrow +TE'_1$	$\{E'.node = E'_1.node;$ $E'_1.i = newNode('+', E'.i, T.node)\}$
$E' \rightarrow -TE'_1$	$\{E'.node = E'_1.node;$ $E'_1.i = newNode('-', E'.i, T.node)\}$
$E' \rightarrow \epsilon$	$\{E'.node = E'.i\}$
$T \rightarrow (E)$	$\{T.node = E.node\}$
$T \rightarrow id$	$\{T.node = newLeaf(id, id.entry)\}$
$T \rightarrow num$	$\{T.node = newLeaf(num, num.lexval)\}$

64 / 85

Abstract syntax trees

ARITHMETIC EXPRESSIONS: LL GRAMMAR

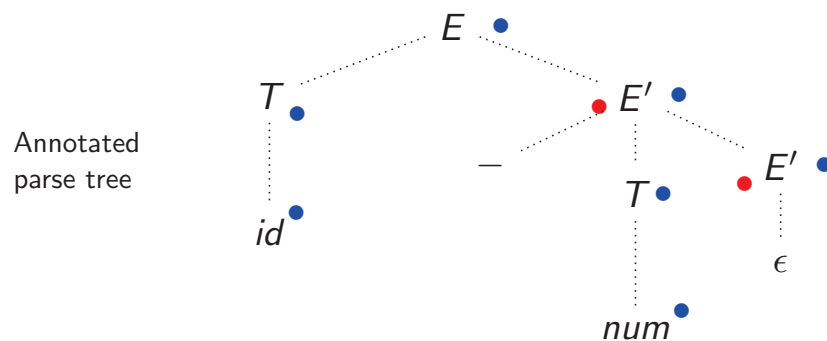
Parse *id* — *num* resulting from lexing *a* — 4



65 / 85

Abstract syntax trees

ARITHMETIC EXPRESSIONS: LL GRAMMAR

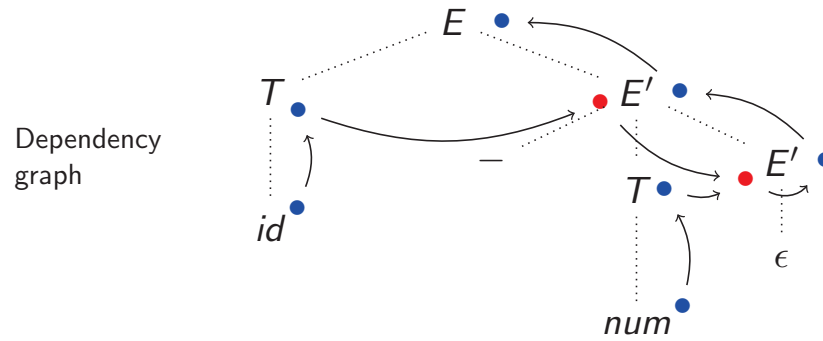


Where ● stands for the inherited attribute of the tree node and ● stands for its synthesized attribute

66 / 85

Abstract syntax trees

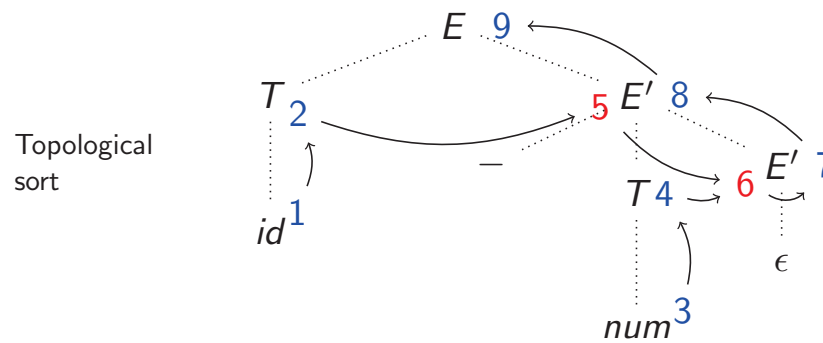
ARITHMETIC EXPRESSIONS: LL GRAMMAR



67 / 85

Abstract syntax trees

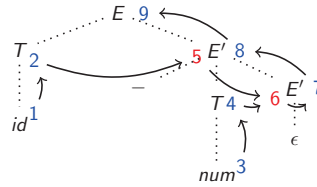
ARITHMETIC EXPRESSIONS: LL GRAMMAR



68 / 85

Abstract syntax trees

ARITHMETIC EXPRESSIONS: LL GRAMMAR



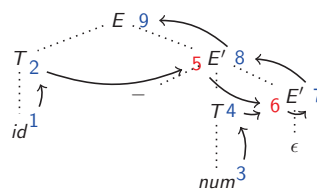
Evaluation:

- 1: $id.entry$ (reference to the entry for a in the symbol table)
- 2: $T.node = newLeaf(id, id.entry)$ with $id.entry$ as given at 1 (which we denote $id.entry\#1$)

69 / 85

Abstract syntax trees

ARITHMETIC EXPRESSIONS: LL GRAMMAR



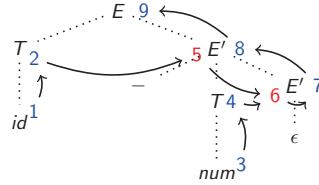
- 3: 4 from $num.lexval = 4$

- 4: $T.node = newLeaf(num, 4)$ with 4 as given at 3

70 / 85

Abstract syntax trees

ARITHMETIC EXPRESSIONS: LL GRAMMAR



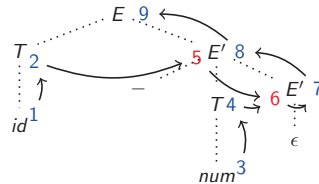
5: $E'.i = T.\text{node}\#2$

6: $E'.i = \text{newNode}('-', E'.i\#5, T.\text{node}\#4)$

71 / 85

Abstract syntax trees

ARITHMETIC EXPRESSIONS: LL GRAMMAR



7: $E'.\text{node} = E'.i\#6$
By $E'.\text{node} = E'.i$ for $E' \rightarrow \epsilon$

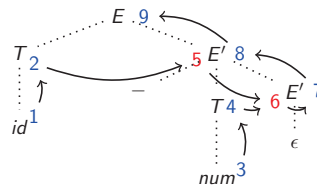
8: $E'.\text{node} = E'.i\#6$
By $E'.\text{node} = E'_1.\text{node}$ for $E' \rightarrow -TE'_1$

9: $E.\text{node} = E'.i\#6$
By $E.\text{node} = E'.\text{node}$ for $E \rightarrow TE'$

72 / 85

Abstract syntax trees

ARITHMETIC EXPRESSIONS: LL GRAMMAR



In overall:

$E.node$ is a reference to $Node('-', E'.i\#5, T.node\#4)$

Where

- $E'.i\#5$: is a reference to $Leaf(id, \text{entry for } a)$
- $T.node\#4$: is a reference to $Leaf(num, 4)$

73 / 85

Symbol tables

Major data structures in a compiler after syntax trees

Major inherited attributes

Typical dictionary data structures

Main operations:

- *insert*
- *lookup*
- *delete*

Typical implementations include: linear lists, search trees, hash tables

Hash tables often the best choice: the main operations can be performed almost in linear time

74 / 85

Hash tables

COLLISION RESOLUTION

Open addressing: Each bucket has enough space for a single item, insert colliding items in successive buckets

Contents of the table limited by the size of the bucket array

Poor performance

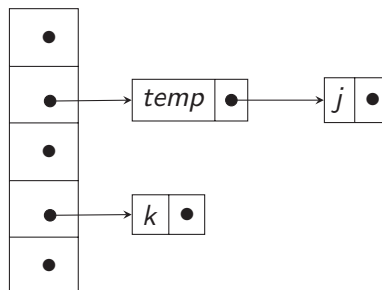
75 / 85

Hash tables

COLLISION RESOLUTION

Separate chaining: Each bucket is a linear list, and hash collisions are resolved by inserting the new item into the list

For example, if *temp* and *j* have the same hash value, then:



76 / 85

Hash tables

HASH FUNCTIONS

Goal: Convert a character string (the name of the identifier) into an integer in the range $0 \dots size - 1$ where *size* is the size of the table and of the bucket array

Tactics:

- Convert each character to a nonnegative integer, usually by built-in conversion mechanisms of the compiler implementation language
- Apply an adequate hash function h (often variable names assigned in groups: *temp1*, *temp2*, ...; take all the characters to avoid too many collisions)
- Typical good choice: $h = (\sum_{i=1}^n \phi^{n-i} c_i) \bmod size$
- Where c_i is the numeric value of the i th character, and ϕ is a power of 2, so that multiplication can be a shift

77 / 85

Hash tables

SCOPE

Names have a declaration (constant declarations, variable declarations, function declarations)

- Local declarations: Names have a limited scope (are visible in a portion of the program)
- Global declarations: Names are visible in the entire program

The same name can be declared several times, then the use of the name refers to its closest declaration

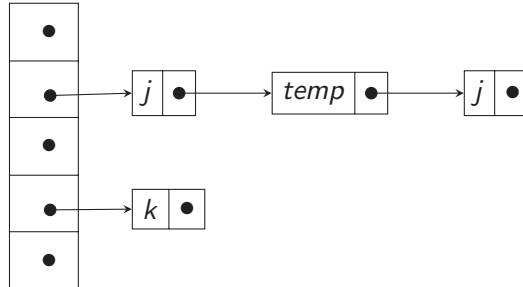
The **scope** of a declaration is a sub-tree of the syntax tree

Nested declarations give rise to scopes that are nested sub-trees of the syntax tree

78 / 85

Hash tables

NESTED SCOPE

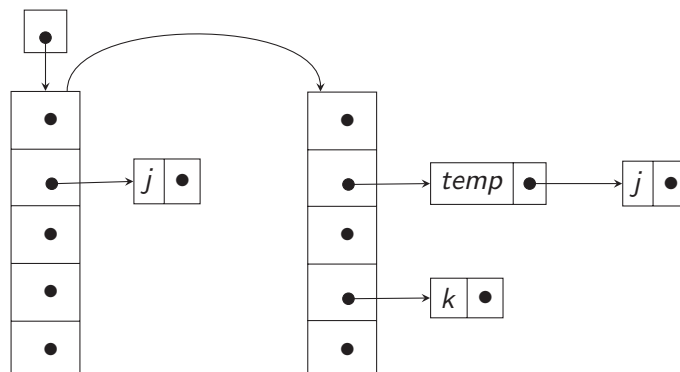


One solution is to manage the hash table in a stack-like manner

79 / 85

Hash tables

NESTED SCOPE



Another solution is to build a new symbol table for each scope and to link the tables from inner to outer scopes together

80 / 85

Training on semantic analysis

81 / 85

Insert types in the symbol table

TRAINING

Design semantic actions to get an SDD for adding the type of the declared identifiers (*int* or *float*) to their entries in the symbol table

$$D \rightarrow TL$$

$$T \rightarrow \textit{int}$$

$$T \rightarrow \textit{float}$$

$$L \rightarrow L_1, \textit{id}$$

$$L \rightarrow \textit{id}$$

To update the table, use function `addtype(table_entry, type_instance)` where *type_instance* is either *int* or *float*

82 / 85

Insert types in the symbol table

TRAINING

$D \rightarrow TL$	$\{L.i = T.type\}$
$T \rightarrow int$	$\{T.type = int\}$
$T \rightarrow float$	$\{T.type = float\}$
$L \rightarrow L_1, id$	$\{L_1.i = L.i; \text{addtype}(id.entry, L.i)\}$
$L \rightarrow id$	$\{\text{addtype}(id.entry, L.i)\}$

83 / 85

Moving on a grid

TRAINING

Design an S-attributed SDD such that, given a string z over the alphabet $\{up, down, left, right\}$, prints the coordinates of the Cartesian plane reached from the origin of the plane and moving: one unit *up* for each *up* in z ; one unit *down* for each *down* in z ; one unit *left* for each *left* in z ; one unit *right* for each *right* in z

84 / 85

Moving on a grid

TRAINING

$E \rightarrow S$	$\{print(S.x, S.y)\}$
$S \rightarrow \epsilon$	$\{S.x = 0; S.y = 0\}$
$S \rightarrow S_1 M$	$\{S.x = S_1.x + M.x; S.y = S_1.y + M.y\}$
$M \rightarrow up$	$\{M.x = 0; M.y = 1\}$
$M \rightarrow down$	$\{M.x = 0; M.y = -1\}$
$M \rightarrow left$	$\{M.x = -1; M.y = 0\}$
$M \rightarrow right$	$\{M.x = 1; M.y = 0\}$